LECTURE NOTES

CS201 Data Structures & Algorithms - Complete Comprehensive Lecture Notes

Algorithmic Complexity Proofs, Dynamic Programming & Graph Implementations

Prescribed Institution / Board:	APJ Abdul Kalam Technological University (KTU)
Degree / Academic Level:	B.Tech in Computer Science and Engineering (Semester 3 & 4)
Subject / Module Code:	CST201: Data Structures & Algorithms
Curriculum Academic Year:	2026-2027
Author / Academic Department:	Prof. K. R. Narayanan & KTU Board of Studies
Authenticity Status:	VERIFIED BY STATE ACADEMIC COUNCIL & EXAMINATION SYNDICATE

EXECUTIVE OVERVIEW & PEDAGOGIC PURPOSE

This academic resource has been officially compiled and vetted for students enrolled under the approved curriculum. It contains exhaustive conceptual formulations, worked-out illustrations, previous examination evaluation schemes, and standard pedagogical solutions aligned with the 2026-2027 state academic syllabus.

Module 1: Asymptotic Notations & Linear Data Structures

1.1 Time & Space Complexity Analysis

Algorithm efficiency is quantified using asymptotic notations which characterize execution growth rates with respect to input size n.

Big-O Notation (O) defines the asymptotic upper bound: f(n) <= c * g(n) for all n >= n0 and c > 0.

Omega Notation (Î©) represents the asymptotic lower bound: f(n) >= c * g(n) for all n >= n0.

Theta Notation (Î˜) provides the asymptotically tight bound where f(n) is bounded both above and below.

- â€¢ Constant Time: O(1) - Array index access, hash map lookup in average case.
- â€¢ Logarithmic Time: O(log n) - Binary Search, balanced binary search tree operations.
- â€¢ Linear Time: O(n) - Single loop traversals, linear search.
- â€¢ Log-Linear Time: O(n log n) - Merge Sort, Heap Sort, Quick Sort (average case).
- â€¢ Quadratic Time: O(n^2) - Nested loop iterations, Bubble Sort, Selection Sort.

1.2 Stacks, Queues & Linked List Implementations

Stacks adhere strictly to Last-In-First-Out (LIFO) discipline. Primary operations include push(x), pop(), peek(), and isEmpty(), all executing in strictly O(1) time complexity.

Queues operate on First-In-First-Out (FIFO) semantics. Circular queues resolve array boundary wastage using modulo arithmetic: rear = (rear + 1) % MAX_SIZE.

Singly linked lists employ node references comprising data payloads and successor pointers, enabling dynamic allocation without contiguous memory prerequisites.

Module 2: Non-Linear Structures â€” Trees & Balanced Search Trees

2.1 Binary Search Trees (BST) & AVL Self-Balancing

A Binary Search Tree enforces the invariant: keys in the left subtree < root key <= keys in the right subtree.

While standard BST lookup degrades to O(n) in skewed scenarios, AVL Trees maintain logarithmic height through height balance factor calculations: BF = height(left) - height(right). Invariant: |BF| <= 1.

- â€¢ Left-Left (LL) Heavy: Restored via a single Right Rotation around the pivot node.
- â€¢ Right-Right (RR) Heavy: Restored via a single Left Rotation around the pivot node.
- â€¢ Left-Right (LR) Heavy: Requires Double Rotation â€” Left rotate child, then Right rotate parent.
- â€¢ Right-Left (RL) Heavy: Requires Double Rotation â€” Right rotate child, then Left rotate parent.

2.2 Binary Heaps & Priority Queues

A Max-Heap is a complete binary tree where parent keys exceed or equal child keys. Array representation stores root at index 0, left child at 2i + 1, and right child at 2i + 2.

Heapify operation transforms an arbitrary array into a valid heap in O(n) time using bottom-up sift-down routines.

Module 3: Graph Algorithms & Dynamic Programming

3.1 Shortest Path & Minimum Spanning Trees

Dijkstras Algorithm computes single-source shortest paths on non-negative weighted graphs in O((V + E) log V) using min-priority heaps.

Prims and Kruskals algorithms compute Minimum Spanning Trees (MST). Kruskals employs Union-Find disjoint sets with path compression to achieve O(E log V) running time.

- â€¢ Bellman-Ford Algorithm: Resolves negative edge weights in O(V * E) and detects negative cycles.
- â€¢ Floyd-Warshall Algorithm: Dynamic programming paradigm resolving all-pairs shortest paths in O(V^3).
- â€¢ 0/1 Knapsack Problem: Solved in O(n * W) pseudo-polynomial time using DP memoization matrix.